

Set A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

Design a **BurgerHub** class based on the provided tester code and expected output. A **central stock of 500 burgers** is maintained and can be distributed among a **maximum of three branches**. Each **burger costs Tk. 150**.

Tester Code	Expected Output
<pre> public class BurgerHubTester { public static void main(String[] args) { BurgerHub.viewAll() System.out.println("==== 1 ===="); System.out.println("Are 100 burgers available? " + BurgerHub.check(100)); System.out.println("==== 2 ===="); BurgerHub b1 = new BurgerHub("Dhanmondi"); BurgerHub b2 = new BurgerHub("Gulshan"); b1.viewSummary(); System.out.println("==== 3 ===="); b1.sell(5); BurgerHub.add(b1, 400); b1.sell(20); BurgerHub.add(b2, 150); BurgerHub.add(b2,40); b2.sell(15); System.out.println("==== 4 ===="); BurgerHub.viewAll(); } } </pre>	<pre> Total Branches: 0 Central Stock: 500 Total Sell: Tk. 0.0 ==== 1 ==== Are 100 burgers available? true ==== 2 ==== Branch ID: 1 Branch Name: Dhanmondi Branch Stock: 0 Branch Sell: Tk. 0.0 ==== 3 ==== Low in stock Burgers added 20 Burgers sold Cannot add beyond capacity Burgers added 15 Burgers sold ==== 4 ==== Total Branches: 2 Central Stock: 60 Total Sell: Tk. 5250.0 ==== Dhanmondi ==== Branch ID: 1 Branch Name: Dhanmondi Branch Stock: 380 Branch Sell: Tk. 3000.0 ==== Gulshan ==== Branch ID: 2 Branch Name: Gulshan Branch Stock: 25 Branch Sell: Tk. 2250.0 </pre>

Set B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

Design a **BookHive** class based on the provided test code and expected output. The system maintains a **central stock of 300 books** that can be allocated to a **maximum of three outlets**. **Each book is issued at Tk. 35.**

Tester Code	Expected Output
<pre> public class BookHiveTester { public static void main(String[] args) { BookHive.showAll() System.out.println("==== 1 ===="); System.out.println("Are 301 books available? "+ BookHive.inStock(301)); System.out.println("==== 2 ===="); BookHive o1 = new BookHive("Badda"); BookHive o2 = new BookHive("Mirpur"); o1.showDetails(); System.out.println("==== 3 ===="); o1.issue(40); BookHive.allocate(410, o1); BookHive.allocate(100, o1); o1.issue(70); BookHive.allocate(30, o2); o2.issue(20); System.out.println("==== 4 ===="); BookHive.showAll(); } } </pre>	<pre> Total Outlets: 0 Available books in System: 300 Total Earnings: Tk. 0.0 ==== 1 ==== Are 301 books available? false ==== 2 ==== Outlet ID: 1 Outlet Name: Badda Books Assigned: 0 Outlet Earnings: Tk. 0.0 ==== 3 ==== Insufficient books in stock Cannot exceed allocation limit Books allocated 70 Books issued Books allocated 20 Books issued ==== 4 ==== Total Outlets: 2 Available books in System: 170 Total Earnings: Tk. 3150.0 ==== Badda ==== Outlet ID: 1 Outlet Name: Badda Books Assigned: 30 Outlet Earnings: Tk. 2450.0 ==== Mirpur ==== Outlet ID: 2 Outlet Name: Mirpur Books Assigned: 10 Outlet Earnings: Tk. 700.0 </pre>

Solution and Rubric

Tentative Solution for Set-A

```
public class BurgerHub {

    public static int total_branches = 0;
    public static int available_burgers = 500;
    public static double total_all_sell = 0;

    public static BurgerHub branch_list[] = new BurgerHub[3];

    public String branch_name;
    public int branch_id;
    public int total_burgers_available;
    public double total_sell;

    public BurgerHub(String name) {
        branch_name = name;
        branch_list[total_branches++] = this;
        branch_id = total_branches;
    }

    public void sell(int amount) {
        if (total_burgers_available - amount >= 0) {
            total_burgers_available -= amount;
            total_sell += amount * 150;
            total_all_sell += amount * 150;
            System.out.println(amount+" Burgers sold");
        } else {
            System.out.println("Low in stock");
        }
    }

    public static boolean check(int amount) {
        return available_burgers >= amount ;
    }

    public static void add(BurgerHub b, int quantity) {
        if (check(quantity)) {
            b.total_burgers_available += quantity;
            available_burgers -= quantity;
            System.out.println("Burgers added");
        } else {
            System.out.println("Cannot add beyond capacity");
        }
    }

    public void viewSummary() {
```

```

        System.out.println("Branch ID: " + branch_id);
        System.out.println("Branch Name: " + branch_name);
        System.out.println("Branch Stock: " + total_burgers_available);
        System.out.println("Branch Sell: Tk. " + total_sell);
    }

    public static void viewAll() {
        System.out.println("Total Branches: "+total_branches);
        System.out.println("Cenrtal Stock: " + available_burgers);
        System.out.println("Total Sell: Tk. " + total_all_sell);
        for (int i = 0; i < total_branches; i++) {
            System.out.println("==== " + branch_list[i].branch_name + "
====");
            branch_list[i].viewSummary();
        }
    }
}

```

Rubric for Set A

SL	Points to Meet	Marks
1	Proper declaration and implementation of static and instance variables	1.5+0.5
2	Correct implementation of the constructor	1
3	Accurate condition checking and updates to all variables in the sell() method	1.5
4	Correct implementation of the static methods, check() and add(), with proper logic handling	3.5
5	Correct implementation of the output methods, viewSummary() and viewAll(), with proper logic handling	2
Total Marks = 10 Marks		

Tentative Solution for Set-B

```
public class BookHive {

    public static int total_outlets = 0;
    public static int available_books = 300;
    public static double total_earnings = 0;

    public static BookHive outlet_list[] = new BookHive[3];

    public String outlet_name;
    public int outlet_id;
    public int books_in_outlet;
    public double outlet_earnings;

    public BookHive(String name) {
        outlet_name = name;
        outlet_list[total_outlets++] = this;
        outlet_id = total_outlets;
    }

    public void issue(int amount) {
        if (books_in_outlet - amount >= 0) {
            books_in_outlet -= amount;
            outlet_earnings += amount * 35;
            total_earnings += amount * 35;
            System.out.println(amount + " Books issued");
        } else {
            System.out.println("Insufficient books in stock");
        }
    }

    public static boolean inStock(int amount) {
        return available_books >= amount;
    }

    public static void allocate(int quantity, BookHive b) {
        if (inStock(quantity)) {
            b.books_in_outlet += quantity;
            available_books -= quantity;
            System.out.println("Books allocated");
        } else {
            System.out.println("Cannot exceed allocation limit");
        }
    }
}
```

```

public void showDetails() {
    System.out.println("Outlet ID: " + outlet_id);
    System.out.println("Outlet Name: " + outlet_name);
    System.out.println("Books Assigned: " + books_in_outlet);
    System.out.println("Outlet Earnings: Tk. " + outlet_earnings);
}

public static void showAll() {
    System.out.println("Total Outlets: " + total_outlets);
    System.out.println("Available books in System: " + available_books);
    System.out.println("Total Earnings: Tk. " + total_earnings);

    for (int i = 0; i < total_outlets; i++) {
        System.out.println("==== " + outlet_list[i].outlet_name + "
====");
        outlet_list[i].showDetails();
    }
}
}

```

Rubric for Set B

SL	Points to Meet	Marks
1	Proper declaration and implementation of static and instance variables	1.5+0.5
2	Correct implementation of the constructor	1
3	Accurate condition checking and updates to all variables in the issue() method	1.5
4	Correct implementation of the static methods, inStock() and allocate(), with proper logic handling	3.5
5	Correct implementation of the output methods, showDetails() and showAll(), with proper logic handling	2
Total Marks = 10 Marks		